Universidad Católica Andrés Bello – Extensión Guayana
Facultad de Ingeniería
Escuela de Ingeniería Informática
Cátedra: [DATO: nombre exacto de la cátedra de Robótica]
Puerto Ordaz – Estado Bolívar

Perry: brazo robótico de cinco grados de libertad que clasifica cubos por color con visión por computador dentro de un sistema multirobot

Informe preliminar

Docente:

[DATO: docente de la cátedra]

Estudiantes:

Aliendres, Ignacio
Baczek, Cristian
[DATO: resto del equipo de Perry]

Ciudad Guayana, noviembre de 2026

1. Introducción

Los brazos robóticos resuelven tareas repetitivas, como tomar piezas y clasificarlas, con una precisión que no se agota con el uso continuo [FUENTE: cifra de robots industriales instalados, IFR World Robotics]. Cuando las piezas no llegan siempre al mismo lugar, el brazo necesita ver dónde están: la visión por computador permite ubicarlas y decidir cómo tomarlas (Lugo, 2023).

En este trabajo se desarrolló Perry, el brazo de un sistema de tres robots. Dos robots móviles, Phineas y Ferb, recogen cubos de la pista y los dejan en una zona azul frente al brazo; Perry los ve con una cámara cenital, los toma y los lleva a su caja según el color. El brazo es el mismo modelo antropomórfico de cinco grados de libertad que usaron Saad (2015) y Lugo (2023) en la UCAB Guayana.

El informe se organiza en siete secciones. En la sección 2 se plantean el problema y los objetivos; la sección 3 analiza el punto de partida, las alternativas y los requerimientos; la sección 4 presenta el diseño; la sección 5, la implementación y la calibración; la sección 6 reporta las pruebas con sus cifras, y la sección 7 recoge las conclusiones. El código, los registros de cada corrida y las pruebas automatizadas están en el repositorio del proyecto (Aliendres et al., 2026).

2. El problema y la propuesta

2.1. Planteamiento del problema

Lugo (2023) desarrolló un sistema de clasificación con este mismo brazo y la misma cámara (Microsoft LifeCam Studio). Reconocía piezas de jenga de 1,5 × 2,5 × 7,5 cm sobre un área de trabajo de 70 × 70 cm y las llevaba a una caja con un error promedio de 0,12 cm en X y 0,11 cm en Y (Lugo, 2023, p. 53). En el sistema multirobot cambian cuatro condiciones (ver Tabla 1).

Tabla 1: Lo que cambia respecto del sistema de Lugo (2023)

Condición nueva	Problema que plantea
Los cubos los dejan robots, no una persona	Posición y giro impredecibles; pueden llegar pegados
Los cubos azules llegan sobre una zona azul	Separar el cubo del fondo con la cámara
La zona la comparten tres robots	Nadie puede estar en ella al mismo tiempo que otro
Servos con juego y desgaste	La calibración de Lugo no sirve tal cual: hay que medir la propia

Es por esto que se planteó un sistema que, además de clasificar, decide cómo tomar cada cubo según los que tiene alrededor, verifica cada agarre y coordina el uso de la zona con los otros dos robots.

2.2. Objetivos

Objetivo general. Desarrollar un sistema de visión y control para que un brazo robótico de cinco grados de libertad clasifique por color los cubos que entregan dos robots móviles.

Objetivos específicos.

1. Analizar el brazo, su cinemática y el sistema de Lugo (2023) frente a las condiciones del sistema multirobot.
2. Definir los requerimientos funcionales y no funcionales con base en el análisis realizado.
3. Diseñar la visión, el planificador de agarre y la coordinación de la zona según los requerimientos definidos.
4. Construir y calibrar el software del brazo con base en el diseño elaborado.

5. Validar el sistema mediante pruebas de precisión, corridas de clasificación y pruebas automatizadas.

2.3. Alcance y limitaciones

Se contemplaron cubos azules y amarillos de 1,9 cm, dejados dentro de una franja de entrega entre 12 y 19,5 cm del eje del brazo, tomados siempre con la pinza vertical. El sistema se opera desde un panel web y corre en una Raspberry Pi 4. Del lado de Perry se implementó la coordinación de la zona según el contrato de los robots móviles. No se contemplaron el modelo dinámico del brazo, los cubos apilados ni otros colores: el rojo, por regla de la pista, se ignora. La prueba integrada con Phineas y Ferb quedó pendiente [DATO: fecha y resultado si se llega a hacer].

Las limitaciones fueron tres. Primero, los servos no informan su posición y la base tiene unos 7° de juego; se mitigó llegando siempre a la base desde el mismo lado y midiendo la precisión con una prueba propia. Segundo, la cámara cenital no tiene un soporte fijo y se movió dos veces; se mitigó calibrándola con las esquinas de la zona azul, de modo que recalibrar es volver a marcar cuatro puntos. Tercero, la Pi no tiene internet en el laboratorio; la imagen de Docker se construye en una PC y se pasa por cable.

3. Análisis

3.1. Punto de partida

La cinemática inversa obtiene los ángulos de las articulaciones a partir de la posición que se quiere para el extremo del brazo (Barrientos et al., 2007). Lugo (2023, pp. 38–40) la resolvió con geometría: con la pinza perpendicular a la mesa, el brazo forma un triángulo rectángulo y uno obtuso; la ley del coseno da el codo y las sumas de ángulos dan el hombro y la muñeca. La base sale de $\theta_1 = \text{atan}(y/x)$. Para enviar los ángulos a la tarjeta Pololu Maestro, que solo recibe pulsos (Pololu Corporation, s. f.), los convirtió con una recta entre dos puntos conocidos de cada servo. Perry reutilizó ese modelo y esa conversión, con las medidas de su propio brazo (ver Figura 2).

3.2. Alternativas evaluadas

Teniendo el problema delimitado, se compararon alternativas en cinco decisiones (ver Tabla 2). La de más peso fue la primera: los ángulos de Lugo son exactos, pero la conversión a pulsos medida con fotos tenía errores de unos 3°, que con el brazo estirado mueven la punta cerca de 1 cm.

Tabla 2: Decisiones de diseño

Decisión	Alternativas	Elegida y por qué
Cómo llegar al cubo	Cinemática de Lugo con ángulos convertidos a pulsos; tabla enseñada	Tabla enseñada: pulsos medidos con la pinza agarrando de verdad a cada distancia. Absorbe la calibración y el resorte del hombro. La cinemática se conservó para la base y como referencia
Encontrar los cubos	Sustracción de fondo; color en HSV	HSV con un rango que excluye el azul de la zona (fondo H 103–104; cubos H 113–124). Más simple; la sustracción queda si cambia la luz
Cubos pegados del mismo color	Separar por forma; por área y k-means	Área y k-means (Lloyd, 1982): la cantidad sale del área; funciona con filas, en L y con giros distintos
Cubo sin espacio para los dedos	Arrastrarlo apoyando la pinza; empujarlo con la pinza cerrada	Empujar: arrastrar patinaba y forzaba el hombro
Comunicación con los carros	MQTT; HTTP; ROS 2	ROS 2: es el contrato que ya programaron los carros. En la Pi (Debian 13) corre en Docker

3.3. Requerimientos

Con base en el análisis se definieron los requerimientos de la Tabla 3. Cada no funcional lleva una cifra para poder verificarlo en las pruebas.

Tabla 3: Requerimientos principales

ID	Requerimiento
RF-01	El sistema debe detectar los cubos azules y amarillos de la zona y calcular su posición en cm y su giro.
RF-02	El sistema debe llevar cada cubo a su caja: azul a la amarilla y amarillo a la verde.
RF-03	El sistema debe ignorar y avisar los cubos que están fuera de su alcance.
RF-04	El sistema debe separar empujando los cubos que no dejan espacio para los dedos.
RF-05	El sistema debe detectar un agarre fallido y reintentarlo una vez.
RF-06	El operador debe poder iniciar, ir paso a paso, solo mirar, cancelar y elegir un color desde un panel web.
RF-07	El sistema debe registrar cada ciclo (fotos, decisión y resultado) para poder repetirlo.
RF-08	El sistema debe coordinar la zona con los carros: llave, espera y cierre.
RNF-01	El error medio de posicionamiento debe ser menor o igual a 5 mm.
RNF-02	Tras una entrega, el brazo no debe moverse antes de 2 s ni con la zona ocupada o en movimiento.
RNF-03	El sistema debe funcionar en una Raspberry Pi 4 sin internet.
RNF-04	Toda medida y todo pulso deben estar en un único archivo de configuración.
RES-01	Python; tarjeta Pololu Mini Maestro 12; cámara cenital de 640 × 480 a 70 cm.
RES-02	ROS 2 Lyrical con los mensajes de multirobot_interfaces (Vielma, 2026).

4. Diseño de la solución

4.1. Coordenadas y ángulos

Teniendo los requerimientos definidos, se fijó un único sistema de coordenadas: el origen está en la mesa, bajo el eje de giro de la base; X apunta al centro de la zona, Y a la izquierda y Z hacia arriba (ver Figura 1). La franja verde es la parte de la zona que el brazo alcanza: entre 11,9 y 19,5 cm del eje. La franja del fondo de la zona azul queda fuera de alcance.

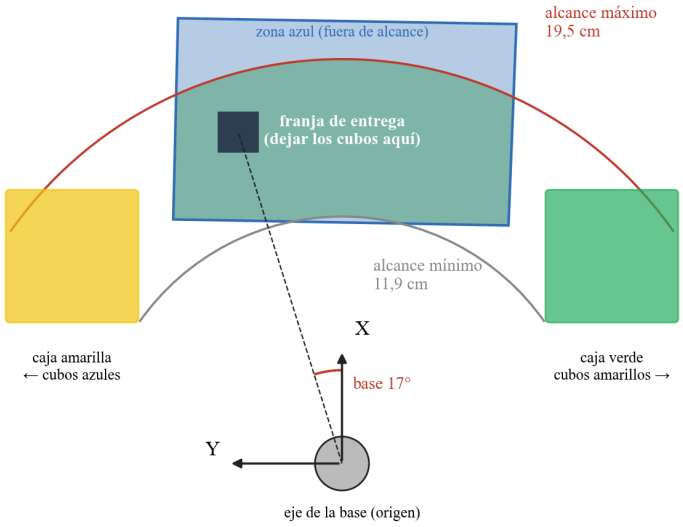


Figura 1: Zona de entrega, alcance del brazo y cajas, vistos desde arriba (cm). Esquinas de la zona tomadas de la calibración de la cámara.

Los ángulos siguen las convenciones de la calibración (ver Figura 2): la base mide 0° sobre el eje X y es positiva hacia la izquierda; el hombro se mide desde la horizontal (90° es vertical); el codo y la muñeca son ángulos interiores (180° es recto), y el giro de la pinza es 0° cuando los dedos cierran de izquierda a derecha. Las medidas del brazo son E1 = 3,3 cm (mesa a hombro), E2 = 10 cm, E3 = 12 cm y E45 = 10,13 cm (muñeca a la punta de la pinza).

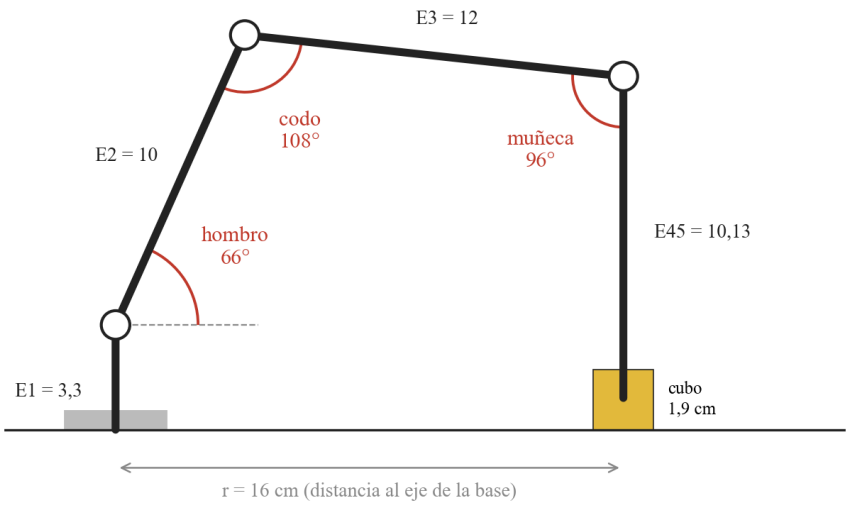


Figura 2: Postura calculada con la cinemática de Lugo (2023) y las medidas de Perry para tomar un cubo a 16 cm.

4.2. Arquitectura del ciclo

Cada cubo recorre cinco etapas (ver Figura 3). La cámara saca la foto recién cuando la escena está quieta, para que el brazo no aparezca saliendo de la imagen. La visión convierte píxeles en centímetros con una homografía calculada con las cuatro esquinas de la zona (Hartley y Zisserman, 2004), detecta los cubos por color, separa los pegados y corrige la paralaje: a 70 cm, la cámara ve la cara de arriba del cubo corrida hacia afuera en la proporción 1,9 / 70. El planificador decide si agarrar, empujar o terminar. El brazo llega por arriba, siempre a la base desde el mismo lado, y baja el hombro despacio. La verificación cuenta los cubos: el agarre salió si hay uno menos de ese color.

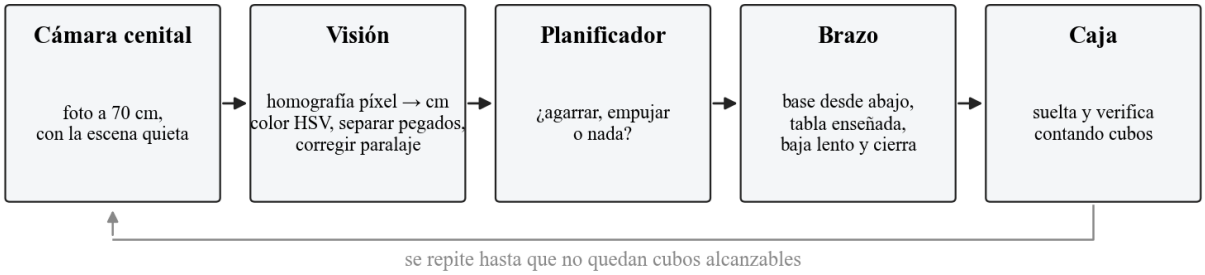


Figura 3: Recorrido de un cubo, de la cámara a la caja.

4.3. Planificador de agarre

Un cubo se puede tomar cerrando la pinza en dos direcciones: según su giro o 90° más. Para cada una se dibujan las huellas de los dos dedos (0,5 × 1,9 cm, más 0,5 cm de error a lo largo del cierre). El eje está libre si ninguna huella pisa otro cubo y si entre los dedos no puede quedar un cubo de otro color. Entre los ejes libres se elige el que menos gira la pinza y el más cercano al brazo. Si ningún cubo tiene un eje libre, se planifica un empuje de 3,8 cm, probando direcciones cada 30°, y se exige que todo cubo movido quede a 1 cm o más de los dos bordes del alcance. Tras tres empujes seguidos sin poder agarrar, la zona se declara bloqueada.

4.4. Coordinación de la zona

La zona se reparte con una sola llave para los tres robots. El árbitro corre en la Pi de Perry: un carro pide la llave (/zone/request), entra, deja el cubo y la devuelve (/zone/release); el árbitro publica dos veces por segundo quién la tiene (/zone/occupant), y si esa señal se corta 2 s, los carros se frenan. Perry pide la misma llave antes de mover el brazo. Que un carro devuelva la llave no prueba que se fue: antes de bajar el brazo, Perry espera 2 s, una escena quieta y ningún objeto más grande que 20 cm² en la zona; si en 10 s no se cumple, la cierra (ver Figura 4). La zona también se cierra con dos cubos esperando, la llave se da por orden de llegada y Perry la cede después de un cubo si un carro espera.

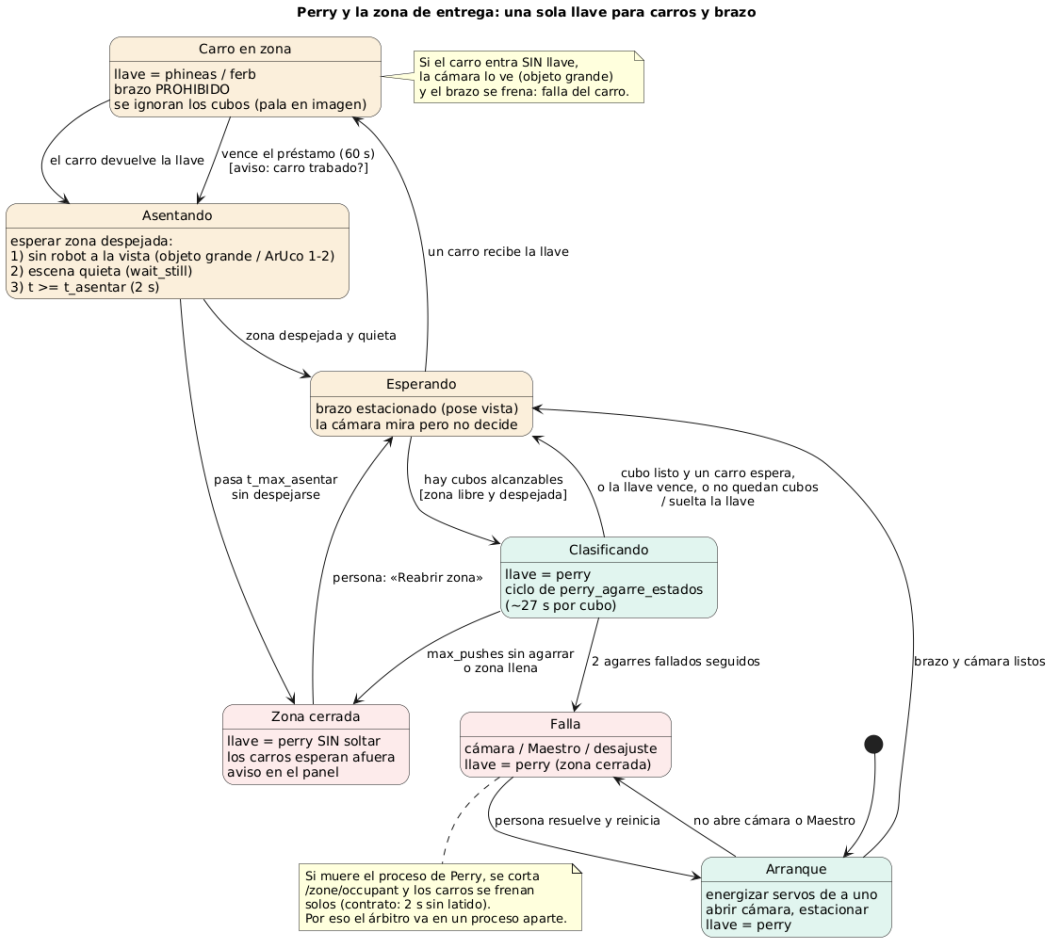


Figura 4: Estados de Perry frente a la zona de entrega.

5. Implementación

5.1. Tecnologías

Habiendo definido el diseño, se construyó todo en Python; las bibliotecas pesadas de imagen están escritas en C++ por dentro. La Tabla 4 muestra las versiones usadas.

Tabla 4: Tecnologías y versiones

Elemento	Versión	Uso
Python	3.13.5 (PC y Pi)	Todo el software
OpenCV / NumPy	4.11.0 / 2.2.6 (PC); 4.10.0 / 2.2.4 (Pi)	Visión y geometría
pyserial / PyYAML	3.5 / 6.0.3 (PC); 6.0.2 (Pi)	Maestro y configuración
pytest	9.2.0	Pruebas automatizadas
ROS 2 Lyrical	imagen ros:lyrical-ros-base	Árbitro y mensajes de la flota, en Docker
Raspberry Pi 4	4 GB, Debian 13 (trixie)	Computador de Perry
Pololu Mini Maestro 12	—	Pulsos a los seis servos
Microsoft LifeCam Studio	640 × 480	Cámara cenital a 70 cm

5.2. Calibración

La base se calibró por tramos con cinco puntos tocados con la pinza (de −36,6° a 36,6°) y se corrigió con la prueba de precisión. Para absorber su juego, siempre se llega a su pulso desde 100 µs más abajo. El giro de la pinza vale 0° en 1447 µs, a unos 10 µs/°. Hombro, codo y muñeca se mueven con una tabla enseñada de siete filas, entre 11,9 y 21,2 cm, que se interpola según la distancia del cubo; al agarrar, el hombro baja 30 µs más por el resorte. La pinza abre en 750 µs, cierra en 1700 µs y empuja cerrada del todo en 2350 µs. Todos estos valores están en un único archivo, arm.yaml.

5.3. Panel, registro y repetición

El panel web muestra la cámara en vivo y la decisión dibujada, y tiene los botones Iniciar, Paso a paso (aprobar cada movimiento), Solo mirar, Prueba de precisión, Cancelar y un selector de color. Cada corrida guarda sus fotos, un registro con hora y un archivo con cada decisión; con él, una corrida se repite con el planificador actual, y una falla entendida en la mesa se convierte en una prueba automatizada.

5.4. Raspberry Pi y Docker

El software se llevó a la Pi por SSH sobre un cable directo. Se encontró que la Pi no arranca si la Maestro y la cámara están conectadas al encender, por lo que se conectan después. Para la flota, el panel corre en un contenedor con ROS 2 Lyrical, con red del anfitrión; la imagen se construye en una PC emulando el procesador de la Pi y se carga por cable [DATO: verificar en el laboratorio].

6. Pruebas

6.1. Precisión

Una vez calibrado el brazo, se midió su precisión con un método propio, porque dejar un cubo y volver a tomarlo en el mismo punto cancela el error. El brazo baja la pinza abierta sobre el cubo, cierra sin levantar y abre: los dedos centran el cubo donde la pinza realmente bajó, y la cámara mide cuánto se corrió. Se repite girando la pinza 90° para obtener X e Y, en cinco puntos (ver Tabla 5).

Tabla 5: Prueba de precisión (31-10-2026, cinco puntos)

Momento	Error medio	Observación
Antes de recalibrar la base	6,4 mm	La base se quedaba corta hasta 2,9° hacia la izquierda
Después de recalibrar la base	3,0 mm	Peor punto: 4,2 mm
Tras un golpe contra una caja	7,9 mm (3,7 sin un punto)	Un cubo giró al cerrar (15,9 mm): desde entonces se descarta la medición si el cubo gira más de 15°
Lugo (2023)	1,2 mm en X; 1,1 mm en Y	Otro método: objeto dejado sobre hoja milimetrada

El error de 3,0 mm cumple RNF-01. Es mayor que el de Lugo (2023), y la diferencia se explica en parte por el método: el suyo mide dónde queda el objeto; el de Perry, dónde baja la pinza, medido con una cámara cuya propia calibración tiene unos 3 mm de precisión. Con los dos métodos en el mismo brazo se podría separar cuánto aporta cada uno.

6.2. Corridas de clasificación

Con la precisión verificada, se hicieron corridas completas con cubos reales, en la PC y en la Raspberry Pi (ver Tabla 6).

Tabla 6: Corridas de clasificación

Corrida	Resultado
PC, 31-10	4 de 4 cubos al primer intento; 2 ignorados por estar fuera de alcance (19,5 y 20,5 cm).
Raspberry Pi, 31-10	5 cubos clasificados; 1 agarre fallido, recuperado al ciclo siguiente; 1 cubo empujado fuera de alcance (11,7 cm), corregido con el margen de 1 cm en ambos bordes.
Tiempo por cubo	Entre 22 y 34 s, mediana 27 s (18 agarres del 30 y 31-10); igual en la PC y en la Pi.
Carga de la Pi	Sin caídas de tensión en 10 min; 46 °C, carga 1,2 de 4 núcleos y 344 MB de memoria como máximo.

6.3. Pruebas automatizadas

Se escribieron 138 pruebas con pytest, que corren en la PC y en la Pi. Incluyen escenas reales guardadas de corridas que fallaron, que desde entonces no pueden volver a fallar sin que se note. El detector de objetos grandes se probó con fotos reales de la cámara cenital (ver Tabla 7).

Tabla 7: Detector de objeto grande con fotos reales (umbral: 20 cm²)

Foto	Mancha mayor	Resultado
Zona vacía, brazo estacionado	0,3 cm²	Zona despejada
Un cubo amarillo / un cubo azul	5,3 / 4,1 cm²	Zona despejada
Brazo sobre la zona	61 cm²	Objeto grande: no se mueve el brazo

6.4. Incidencias

Cada falla vista en la mesa se registró con su causa y su solución (ver Tabla 8).

Tabla 8: Incidencias y soluciones

Incidencia	Solución
La pinza tomaba los cubos por los vértices	Se corrigió el signo del giro de la pinza
Al empujar, la pinza caía sobre el cubo	Empujar con la pinza cerrada del todo (2350 µs)
La pose de espera izquierda golpeaba la caja amarilla	Se desactivó; el brazo espera a la derecha
La foto salía con el brazo cruzando la zona	Esperar dos cuadros seguidos casi iguales
El fondo azul se detectaba como cubo	El rango azul empieza en H 109
Un empuje dejó un cubo fuera de alcance	Margen de 1 cm en los dos bordes, con prueba
La Pi no arrancaba	Conectar la Maestro y la cámara después de encenderla

7. Conclusiones

Una vez construido el sistema y ejecutadas las pruebas, se concluye que:

1. El modelo geométrico de Lugo (2023) siguió siendo válido para este brazo, pero no su conversión a pulsos: medir los pulsos con la pinza agarrando de verdad resultó más preciso que convertir ángulos exactos con una calibración aproximada.
2. Recalibrar la base con la prueba de precisión bajó el error medio de 6,4 a 3,0 mm, por debajo del umbral de 5 mm. Medir antes de ajustar evitó corregir lo que no estaba mal: tras el golpe, el error alto venía de una medición defectuosa y no del brazo.

3. Planificar cada agarre según los cubos vecinos permitió tomar cubos pegados y girados, y empujar solo cuando no había otra salida. En la Pi se clasificaron cinco cubos con un solo agarre fallido, recuperado.
4. Calibrar la cámara con las esquinas de la zona, en lugar de una hoja que se retira, convirtió la recalibración en marcar cuatro puntos.
5. Guardar cada corrida y convertir sus fallas en pruebas automatizadas permitió corregir sin romper lo que ya funcionaba: 138 pruebas protegen el sistema.
6. Usar la misma llave para carros y brazo, y no confiar solo en ella antes de mover el brazo, protegió el caso más peligroso: un carro detenido dentro de la zona.
7. La Raspberry Pi 4 no fue el cuello de botella: el tiempo por cubo fue el mismo que en la PC, y lo marca la mecánica del brazo.

Con base en estas conclusiones se recomienda: aplicar el método de medición de Lugo (2023) en este brazo para comparar ambos con la misma referencia; fijar la cámara con un soporte rígido; medir el umbral de objeto grande con la pala real de cada carro; reemplazar los servos con más juego, y completar la prueba integrada con Phineas y Ferb.

Referencias bibliográficas

Aliendres, I., Baczek, C. y [DATO: resto del equipo] (2026). *Robot_Claw* [Repositorio de código fuente]. GitHub. https://github.com/Kobalt09/Robot_Claw

Barrientos, A., Peñín, L. F., Balaguer, C. y Aracil, R. (2007). *Fundamentos de robótica* (2.ª ed.). McGraw-Hill.

Hartley, R. y Zisserman, A. (2004). *Multiple view geometry in computer vision* (2.ª ed.). Cambridge University Press.

Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory, 28*(2), 129–137.

Lugo, M. (2023). *Sistema de clasificación basado en visión por computador con brazo robótico antropomórfico de 5 grados de libertad* [Trabajo de grado]. Universidad Católica Andrés Bello.

Pololu Corporation. (s. f.). *Maestro servo controller user's guide*. https://www.pololu.com/docs/0J40

Saad, J. (2015). *Entorno de programación para simular y controlar un brazo robótico antropomórfico de 5 grados de libertad* [Trabajo de grado]. Universidad Católica Andrés Bello.

Vielma, A. (2026). *ucab-multirobot* [Repositorio de código fuente]. GitHub. https://github.com/alejandrovielma/ucab-multirobot